

Repix — CPU model comparison

Real-ESRGAN upscaling and DDColor-Tiny colourisation, ONNX Runtime CPU · measured at the deployment target: Hostinger KVM 2 (2 vCPU, 8 GB RAM) · 19 Sep 2026

Summary

- **Speed at the target spec.** A $\times 4$ upscale of $512 \times 512 \rightarrow 2048 \times 2048$ takes **69.6 s**; $\times 2$ of 512×512 takes 11.7 s; a $1024 \times 1024 \rightarrow 4096 \times 4096$ $\times 4$ job takes 4 m 55 s; colourising one image takes 2.2 s (fixed cost). Throughput is ≈ 76 GMAC/s (§4).
- **Memory.** Peak RAM of the shipped configuration is 1143 MiB for $\times 4$ up to 4096^2 output — under 20 % of the 8 GB plan. A measured linear model predicts it from tile size (§5.3).
- **Caps held.** Across 35 timed runs the container never used more than 2.00 cores (cap 2.0) or 3262 MiB (cap 8192 MiB).
- **What helped.** Arena + memory-pattern ON: -26 % time for +59 % RAM on the 600×400 test. On the $1024^2 \rightarrow 4096^2$ job it is 208 s vs 295 s using 3262 MiB vs 1143 MiB.
- **What did not.** Constant-shape tiles: +93 % time on this small image (they recompute edge pixels, §5.1b). Stock ORT defaults on a 2-CPU VPS: +59 % time, +60 % RAM; with only a CPU quota (host CPUs visible) +35 % and 523 s of CPU throttling. Within noise (≈ 6 %): spinning -2 %, thread env -2 %, ORT 1.23.0 0 %. Skipping the NCHWc layout transform: +12 % (slower).
- **Colourisation.** Mean chroma error ΔE^*_{ab} 19.3 vs 28.9 for “no colour” (lower is better). On astronaut the model was worse than adding no colour (§8).
- **Candidate INT8 x4plus:** 40.1 s vs 60.3 s (1.5 $\times$ faster), peak 921 MiB vs 920 MiB — quality in §6.
- **Candidate general-x4v3:** 3.7 s vs 60.3 s (16.3 $\times$ faster), peak 332 MiB vs 920 MiB — quality in §6.

Contents: 1 Target and safety · 2 Models & approach · 3 Math · 4 Baseline performance · 5 Optimisation experiments · 6 Candidate models · 7 Upscaling quality · 8 Colourisation · 9 Recommendations & limitations

1 Deployment target and safety

The app will be hosted on a **Hostinger KVM 2 (2 vCPU, 8 GB RAM)**. The test machine is much bigger (AMD Ryzen 9 8945HS w/ Radeon 780M Graphics, 16 threads, 15 GB, Linux 6.8.0-138-generic, no swap), so every measurement runs in a container limited to the target's resources:

Target resource	Container limit used	Why
2 vCPU	<code>--cpus 2 --cpuset-cpus 0,1</code>	quota + pinning to two physical cores: the app sees exactly 2 CPUs, as it would on the VPS
8 GB RAM, no swap	<code>--memory 8g --memory-swap 8g</code>	hard cap; overshoot means the container is OOM-killed, never the host
—	<code>--pids-limit 256 --network none --read-only --cap-drop ALL</code>	no fork bombs, no network, nothing writable except a scratch mount
—	<code>--oom-score-adj 800</code>	if the host ever ran short, the kernel kills the benchmark first

Extra guards in the runner (stricter than the cap, because this machine is shared with a desktop): one container at a time; refuses to start with < 6 GB free host RAM or load > 8 ; a watchdog kills the container if host free RAM drops under 2 GB or a run exceeds its timeout; the script cannot request more than 2 CPUs / 8 GB.

Measured inside every run (cgroup accounting)	Value	Cap
Timed calls (all runs)	35	
Max average cores used (CPU-seconds $\div$ wall-seconds)	2.00	2.00
Max container RAM (sampled every 20 ms)	3262 MiB	8192 MiB
Runs OOM-killed / stopped by watchdog	0	—

Protocol: fresh container per configuration; model loaded once (not timed); the last of 1–3 calls is reported as the warm time; timing covers the engine call only (image $\rightarrow$ array $\rightarrow$ inference with tiling $\rightarrow$ 8-bit image), not upload, PNG encoding, queueing or HTTP.

2 Models and approach

Model	Role	Weights	Params	File	SHA-256
Real-ESRGAN x4plus	x4 upscaler (RRDBNet, 23 blocks)	FP32	16.7 M	64 MiB	5c586662929c
Real-ESRGAN x2plus	x2 upscaler (RRDBNet + pixel-unshuffle)	FP32	16.7 M	64 MiB	7115ba92e8a1
DDColor-Tiny	Colourisation (ConvNeXt + transformer decoder)	FP16 stored, FP32 compute	67.4 M	129 MiB	2653da00dc15

x4 is a dedicated model (never x2 twice). Hashes match models/CHECKSUMS.sha256. DDColor's FP16 initialisers are cast to FP32 by ONNX Runtime on CPU.

Upscaling

- RGB float32 → NCHW. Images larger than the tile size are cut into **256×256 tiles with a 16 px halo**; only each tile's non-overlapping core is pasted into the output, so tile-border artefacts are dropped without blending.
- Output is clipped, rounded to 8-bit and saved as PNG (lossless).

Colourisation

- The original luminance L is kept at full resolution. A 512×512 grey copy goes through DDColor, which predicts chroma (a, b); it is resized back and recombined with the untouched L — **brightness and detail are preserved exactly**, only colour is invented.

Optimisations already in the shipped code

Optimisation	Where	Purpose
Threads = CPU quota / cgroup cpu.max + affinity)	core/config.py	ORT would otherwise size its pool from host cores
Spinning off, inter-op 1, sequential execution	ort_options.py	idle threads sleep instead of burning quota
CPU memory arena off + memory-pattern off	ort_options.py	avoid pool growth with varying tile shapes
Tiling with fixed tile size	realesrgan.py	bounds memory independent of image size
Bounded queue, MAX_AI_WORKERS = 1	manager.py	extra jobs wait instead of taking CPU

3 The math

3.1 Output size and tiling

$$W_o = sW, \quad H_o = sH, \quad P_o = s^2 WH \quad (s \in \{2, 4\})$$

The app caps output at 50 M px, so the largest x4 input is ≈ 3.1 MP (e.g. 2500×1250).

$$n_x = \lceil W/T \rceil, \quad n_y = \lceil H/T \rceil, \quad N = n_x n_y \quad T = 256, \quad o = 16$$

Every tile carries a halo, so a few pixels are computed twice: $\eta = P_{\text{processed}} / (W \cdot H) - 1$.

Input	Tiles N	Pixels processed	Overlap overhead η	Output (x4)
256×256	1	65,536	0.0 %	1024×1024
512×512	4	295,936	12.9 %	2048×2048
1024×1024	16	1,254,400	19.6 %	4096×4096
1920×1080	40	2,589,952	24.9 %	7680×4320
2500×1250	50	3,841,864	22.9 %	10000×5000

An input that fits one tile ($\max(W, H) \leq 256$) runs in a single pass without overhead.

3.2 Compute cost of Real-ESRGAN (RRDBNet)

One dense block (RDB) has five 3×3 convolutions with 64→32, 96→32, 128→32, 160→32, 192→64 channels; one MAC per weight per output position:

$$\text{MAC}_{\text{RDB}} = 9(64 \cdot 32 + 96 \cdot 32 + 128 \cdot 32 + 160 \cdot 32 + 192 \cdot 64) = 239,616$$

The trunk is 23 RRDB × 3 RDB = 69 RDBs → 16,533,504 MAC/px at the input grid. Adding the remaining 64-channel convolutions (the upsampling convs run at 2× and 4× the resolution, i.e. 4× and 16× the pixels) gives the x4 network:

$$c_4 = 16,533,504 + 36,864 + 1,728 + 4 \cdot 36,864 + 16(2 \cdot 36,864 + 1,728) = 17,926,848 \text{ MAC/px}$$

The $\times 2$ network folds each 2×2 block into channels first (pixel-unshuffle $\rightarrow$ 12 channels at $\frac{1}{4}$ of the pixels), making it $4\times$ cheaper per input pixel:

$$c_2 = \frac{16,533,504 + 36,864 + 6,912 + 4 \cdot 36,864 + 16(2 \cdot 36,864 + 1,728)}{4} = 4,483,008 \text{ MAC/px}$$

Check against the model file: summing every Conv node of the ONNX graphs gives 73,428,369,408 and 18,362,400,768 MAC at 64×64 — exactly $64^2 \cdot c_4$ and $64^2 \cdot c_2$ (asserted in code). 1 MAC = 2 FLOP, so $\times 4$ costs 35.9 MFLOP per input pixel.

$$\text{MAC}_{\text{job}} = c_s P_{\text{processed}} = c_s (1 + \eta) WH$$

DDColor-Tiny always runs on 512×512 , so its cost is fixed (≥ 6.6 GMAC from resolved Conv/MatMul nodes — a lower bound, some attention MatMuls have dynamic shapes); only the Lab conversions and resizes scale with $W \cdot H$.

3.3 Time and memory prediction

$$t \approx \frac{\text{MAC}_{\text{job}}}{G}, \quad M_{\text{peak}} \approx M_0 + k \cdot F + 12 W_0 H_0 B, \quad F = 16 A_{\text{tile}} \cdot 64 \cdot 4 B$$

G is the measured throughput (§4); F is one 64-channel FP32 feature map at $\times 4$ output resolution for the largest tile window of A_{tile} pixels ($(T+2o)^2 = 288^2$ for a full interior tile); the last term of M_{peak} is the float32 output canvas of the shipped engine. §5.3 fits $k = 1.93$ and $M_0 = 316$ MiB ($R^2 = 1.000$) from four tile sizes. For a full 288^2 interior tile one feature map is $(4 \cdot 288)^2 \cdot 64 \cdot 4 B = 324$ MiB.

3.4 Quality metrics

$$\text{PSNR} = 10 \log_{10} \frac{255^2}{\text{MSE}}, \quad \text{SSIM}(x, y) = \frac{(2\mu_x \mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

On luma $Y = 0.299R + 0.587G + 0.114B$, 4-px border shaved, Gaussian 11×11 window ($\sigma = 1.5$), $C_1 = (0.01 \cdot 255)^2$, $C_2 = (0.03 \cdot 255)^2$. Each test photo is downsampled by s (INT_AREA, “clean”) or additionally corrupted with $\sigma=2$ noise and JPEG $q=50$ (“degraded”); the upscaled result is compared with the original.

For colourisation (Lab, D65; L is preserved exactly so the error is pure chroma):

$$\Delta E_{ab}^* = \sqrt{(a - \hat{a})^2 + (b - \hat{b})^2}, \quad M = \sqrt{\sigma_{rg}^2 + \sigma_{yb}^2} + 0.3 \sqrt{\mu_{rg}^2 + \mu_{yb}^2}$$

$rg = R - G$, $yb = \frac{1}{2}(R + G) - B$; M is the Hasler-Süsstrunk colourfulness; “baseline ΔE ” is the error of outputting no colour.

4 Baseline performance at the target spec

Shipped configuration, 2 vCPU / 8 GB. Content is the astronaut photo resized to each size (run time depends on pixel count, not content).

Workload	MAC (job)	cold	warm	G (GMAC/s)	MAC per CPU-s	peak RAM
Colourise 512×512	≥ 7 G	2.3 s	2.2 s	—	—	1411 MiB
Colourise 1024×1024	≥ 7 G	2.3 s	2.2 s	—	—	1453 MiB
Upscale ×2: 512 ² → 1024 ²	1,327 G	11.6 s	11.7 s	113	57 G	561 MiB
Upscale ×4: 256 ² → 1024 ²	1,175 G	16.3 s	16.9 s	69	37 G	914 MiB
Upscale ×4: 512 ² → 2048 ²	5,305 G	68.8 s	69.6 s	76	40 G	1036 MiB
Upscale ×4: 1024 ² → 4096 ²	22,487 G	—	4 m 55 s	76	40 G	1143 MiB

cold = first call, warm = last call. G = achieved GMAC/s; “MAC per CPU-s” divides by CPU time actually consumed (efficiency per core). Colourisation is transformer-heavy and its MAC count is only a lower bound, so no throughput is quoted for it. Model load at startup: 0.7 s (both upscalers).

4.1 Where the time goes: per-layer profile of one 288×288 tile (×4)

Part of the network	Time	Share of time	Share of MACs
Trunk: 69 dense-block convs + conv_first + conv_body (LR resolution)	13.16 s	73 %	92.4 %
Tail: upsampling convs at 2× and 4× resolution + resizes	1.23 s	7 %	7.6 %
Layout reorders (NCHWc ⇄ NCHW) inserted by ORT	3.68 s	20 %	0 %

From ONNX Runtime's profiler (second call, single tile, wall 19.3 s). By op: Conv 11.66 s, ReorderInput 3.14 s, Concat 2.41 s, ReorderOutput 0.54 s, Add 0.16 s. The tail takes 7 % of the time for 7.6 % of the MACs, i.e. proportional: the hypothesis that the 4×-resolution tail is a memory-bandwidth bottleneck is **not** supported on this machine. What stands out instead is layout conversion: ORT runs the convolutions in a blocked NCHWc layout and inserts reorders that cost 20 % of kernel time; §5 tests whether skipping that transform pays off.

4.2 What a job costs on this plan (predicted from the measured throughput)

Input	Scale	Tiles	MAC	Predicted time	Predicted peak RAM	Output
1000×750	×4	12	16.0 T	3 m 30 s	1.1 GB	4000×3000
1920×1080	×2	40	11.6 T	1 m 42 s	1.0 GB	3840×2160
1920×1080	×4	40	46.4 T	10 m 09 s	1.3 GB	7680×4320
2500×1250	×4	50	68.9 T	15 m 04 s	1.5 GB	10000×5000

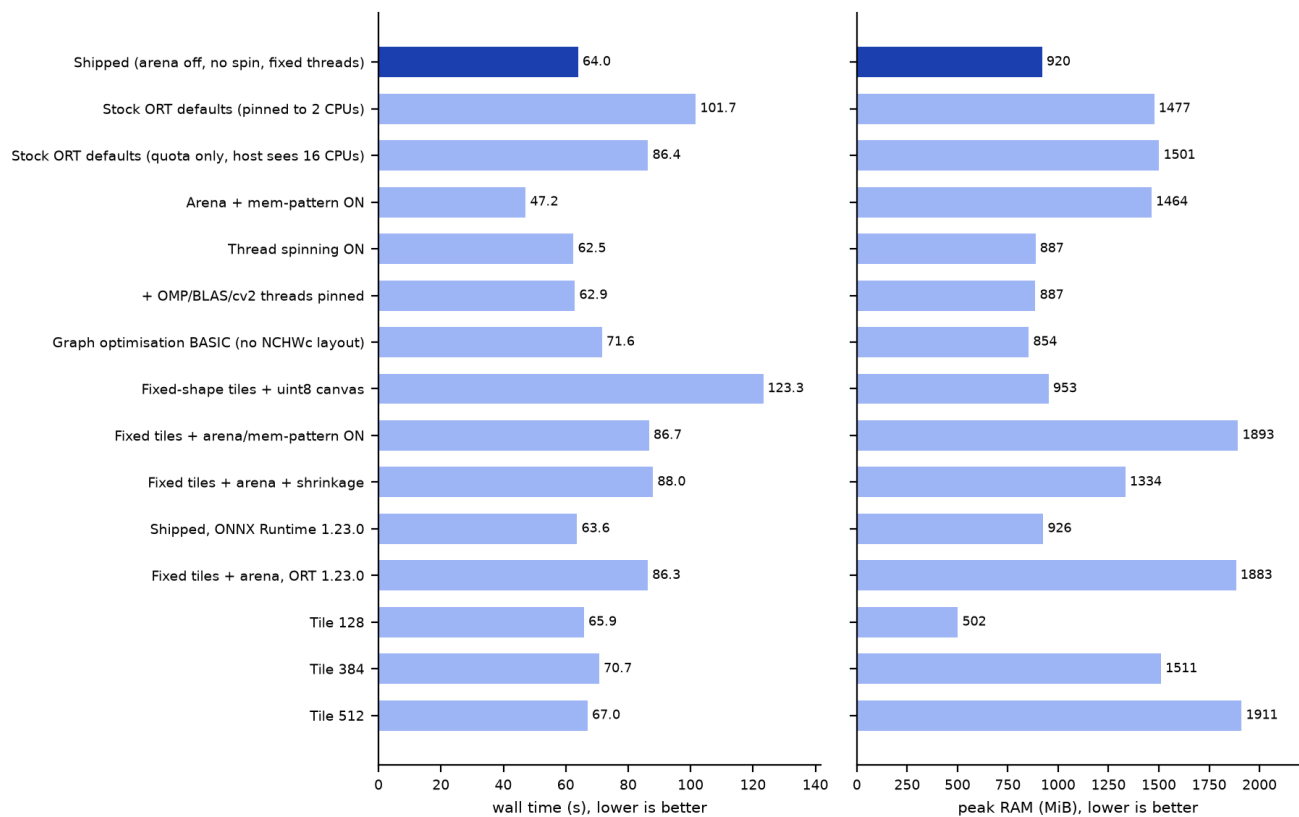
Time = MAC ÷ measured G (×4 uses the ×4 throughput, ×2 the ×2 throughput). The last row is the largest ×4 job the app accepts (50 M output pixels). The memory column uses the fit in §5.3 plus the float canvas and is an extrapolation. Even the largest job is a long-running one: timeouts and the UI must allow for it, and a full queue (5 jobs) can hold more than an hour of work.

5 Optimisation experiments (×4 of a 600×400 photo, 2 vCPU / 8 GB)

Each technique is switched on or off in an otherwise identical fresh container. The 600×400 input is cut into 3×2 tiles of *different* shapes, which is the case the arena and fixed-shape ideas target. Every variant is one cold call; the shipped configuration (the baseline, first call = 64.0 s) was run three times (64.0, 60.8, 60.3 s), a spread of **6.0 %**, so differences below $\approx 6\%$ are treated as noise.

Configuration	Time	Δ time	Peak RAM	Δ RAM	Throttled	Output vs shipped	Status
Shipped (arena off, no spin, fixed threads)	64.0 s		920 MiB		0	—	ok
Stock ORT defaults (pinned to 2 CPUs)	101.7 s	+59 %	1477 MiB	+60 %	0	identical	ok
Stock ORT defaults (quota only, host sees 16 CPUs)	86.4 s	+35 %	1501 MiB	+63 %	523 s	identical	ok
Arena + mem-pattern ON	47.2 s	-26 %	1464 MiB	+59 %	0	identical	ok
Thread spinning ON	62.5 s	-2 %	887 MiB	-4 %	0	identical	ok
+ OMP/BLAS/cv2 threads pinned	62.9 s	-2 %	887 MiB	-4 %	0	identical	ok
Graph optimisation BASIC (no NCHWc layout)	71.6 s	+12 %	854 MiB	-7 %	0	93.3 dB	ok
Fixed-shape tiles + uint8 canvas	123.3 s	+93 %	953 MiB	+4 %	0	49.0 dB	ok
Fixed tiles + arena/mem-pattern ON	86.7 s	+36 %	1893 MiB	+106 %	0	49.0 dB	ok
Fixed tiles + arena + shrinkage	88.0 s	+38 %	1334 MiB	+45 %	0	49.0 dB	ok
Shipped, ONNX Runtime 1.23.0	63.6 s	0 %	926 MiB	+1 %	0	identical	ok
Fixed tiles + arena, ORT 1.23.0	86.3 s	+35 %	1883 MiB	+105 %	0	49.0 dB	ok
Tile 128	65.9 s	+3 %	502 MiB	-45 %	0	48.8 dB	ok
Tile 384	70.7 s	+11 %	1511 MiB	+64 %	0	46.2 dB	ok
Tile 512	67.0 s	+5 %	1911 MiB	+108 %	0	52.4 dB	ok

Δ vs the shipped first (cold) call, negative = better. “Output vs shipped” compares this run's PNG with the shipped run's: “identical” = bit-for-bit the same image, so the option only changes speed/memory; a PSNR figure appears where the tile geometry changes (different context at tile borders; ≥ 45 dB is visually identical). Throttled = seconds the kernel paused the container for exceeding its CPU quota (largest of the runs).



5.1 Reading the results

- **Stock ORT options on a 2-CPU pinned VPS:** +59 % time, +60 % RAM → not recommended (costs more than it saves).
- **Stock ORT options when the container only has a CPU quota (host CPUs visible):** +35 % time, +63 % RAM → not recommended (costs more than it saves).
- **Arena + memory-pattern on:** -26 % time, +59 % RAM → mixed: a trade-off.
- **Thread spinning on:** -2 % time, -4 % RAM → no measurable effect.
- **Pinning OMP/BLAS/OpenCV threads:** -2 % time, -4 % RAM → no measurable effect.
- **Graph optimisation level BASIC instead of ALL (skips the NCHWc layout transform and its reorders, §4.1):** +12 % time, -7 % RAM → not recommended (costs more than it saves).
- **Constant tile shape (edge tiles shifted inward) + uint8 canvas:** +93 % time, +4 % RAM → not recommended (costs more than it saves).
- **Constant tile shape + arena + memory-pattern:** +36 % time, +106 % RAM → not recommended (costs more than it saves).
- **...plus per-run arena shrinkage:** +38 % time, +45 % RAM → not recommended (costs more than it saves).
- **ONNX Runtime 1.23.0 vs 1.20.1:** 0 % time, +1 % RAM → no measurable effect.

5.1b Why constant-shape tiles cost so much on small images

To keep every tile at $(T+2o)^2 = 288^2$, edge tiles are shifted inward, so border pixels are computed twice. The extra work depends on image size (analytic, same tiling model as §3):

Input	Shipped: px processed	Fixed-shape: px processed	Extra compute
600×400	286,848	497,664	+73 %
1024×1024	1,254,400	1,327,104	+6 %
1920×1080	2,589,952	3,317,760	+28 %
2500×1250	3,841,864	4,147,200	+8 %
3000×2000	7,454,848	7,962,624	+7 %

The 600×400 test image is the worst case (it is barely larger than one tile); the measured slow-down of that run follows this table. Padding edge tiles instead would cost the same order of compute.

5.1c Does the arena's memory growth appear on a larger image?

The shipped code comments warn that the arena can keep growing on large jobs. Same job as the \$4 large run, one cold call each, 8 GiB cap:

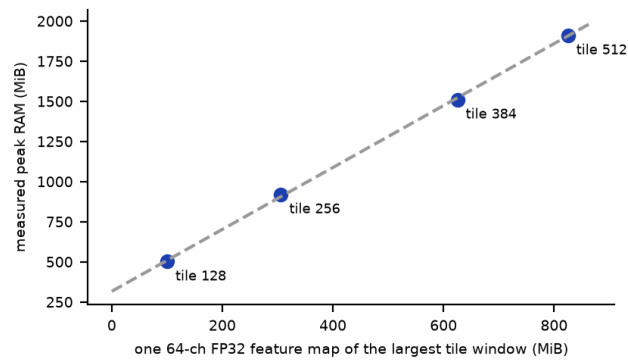
Configuration (×4, 1024 ² → 4096 ²)	Time	Peak RAM	Δ time	Δ RAM	Status
Shipped (arena off)	4 m 55 s	1143 MiB			ok
Arena + mem-pattern ON	3 m 28 s	3262 MiB	-30 %	+185 %	ok
Fixed tiles + arena + shrinkage	3 m 54 s	1386 MiB	-21 %	+21 %	ok

5.2 Tile size

Tile	Tiles	Pixels processed	Time	Peak RAM	Output vs shipped
128	20	361,088	65.9 s	502 MiB	48.8 dB
256 (shipped)	6	286,848	64.0 s	920 MiB	—
384	4	273,024	70.7 s	1511 MiB	46.2 dB
512	2	252,800	67.0 s	1911 MiB	52.4 dB

Smaller tiles re-compute more halo but use less memory; larger tiles amortise the halo but grow memory quadratically.

5.3 Memory model



Peak RAM $\approx 316 \text{ MiB} + 1.93 \times (\text{one 64-channel FP32 feature map of the largest tile window})$, $R^2 = 1.000$ over four tile sizes. $k \approx 2\text{--}4$ means ORT keeps a few full-resolution maps alive at once. This lets the API estimate a job's memory before starting it and reject what cannot fit, rather than finding out from the OOM killer.

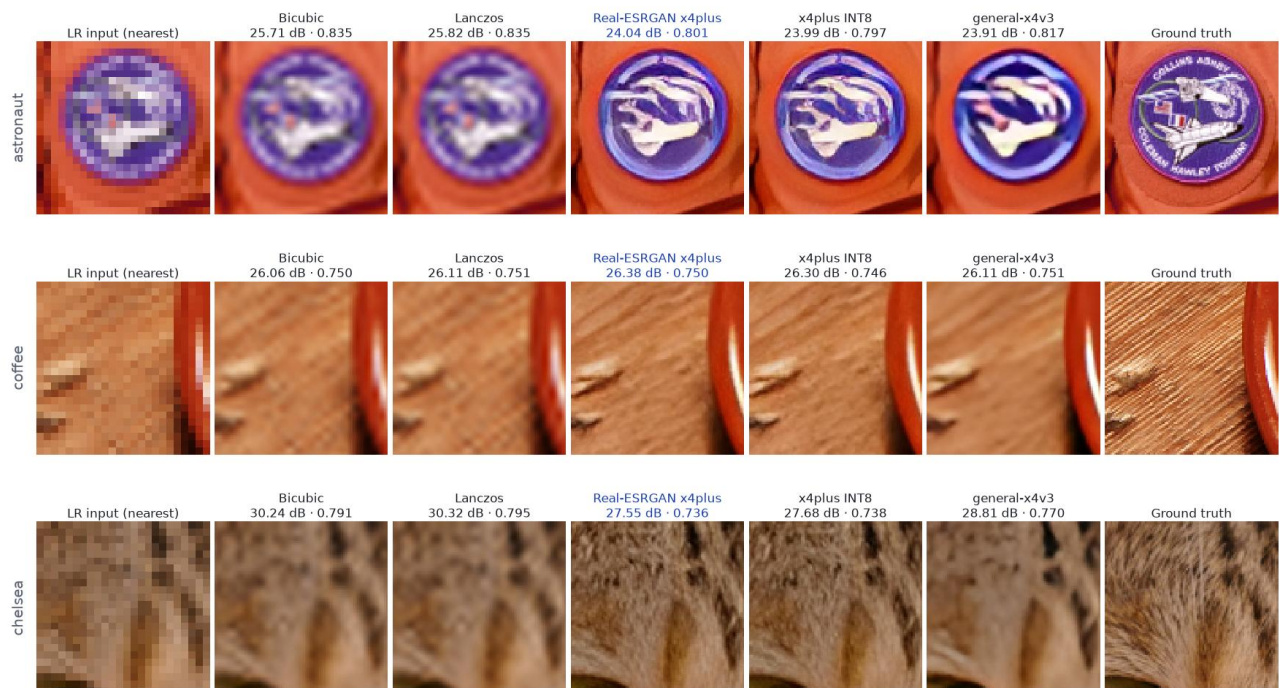
6 Candidate models (not part of the shipped stack)

DECISIONS.md fixes the shipped models; these two are measured only to quantify the trade-off suggested by the research notes. Both use the identical tiling and ORT settings; only the ONNX file differs.

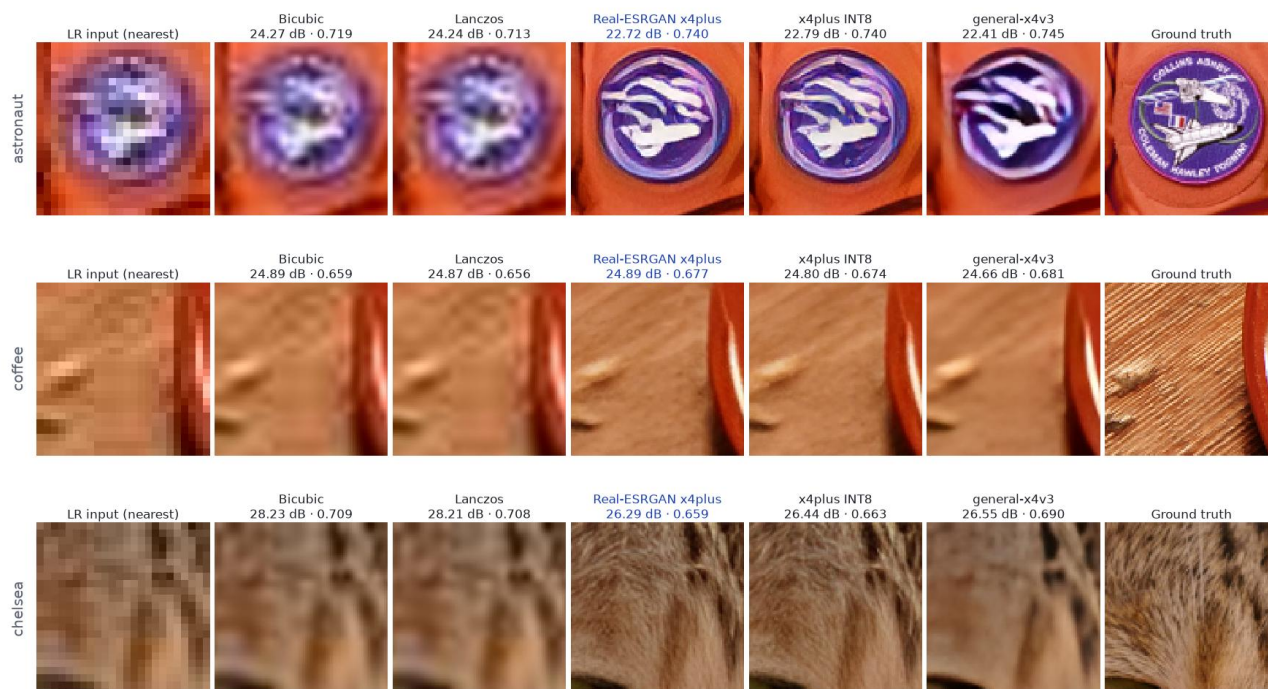
Model	Time (warm)	Speed-up	Peak RAM	Size	Mean Y-PSNR clean	Mean Y-PSNR degraded	Mean SSIM clean / degraded
Real-ESRGAN x4plus FP32 (shipped)	60.3 s	1.0×	920 MiB	64 MiB	25.99 dB	24.63 dB	0.762 / 0.692
x4plus INT8 (static QDQ, per-channel)	40.1 s	1.5×	921 MiB	17 MiB	25.99 dB	24.68 dB	0.761 / 0.692
realesr-general-x4v3 (compact)	3.7 s	16.3×	332 MiB	5 MiB	26.27 dB	24.54 dB	0.780 / 0.705

Speed/RAM: $\times 4$ of the 600×400 photo (same job as §5). Quality: mean over three ground-truth photos (§7). INT8 calibration used 8 held-out images (textures and a rocket photo) that are not in the evaluation set; conv_first and the whole upsampling head are kept in FP32. general-x4v3 is exported from the official checkpoint (BSD-3-Clause per the Real-ESRGAN repository; verify before shipping) and checked against PyTorch.

$\times 4$, clean input — 96×96 crops of the highest-detail region · captions: full-image Y-PSNR / SSIM



x4, degraded input (noise $\sigma=2$ + JPEG q50) — same crops · captions: full-image Y-PSNR / SSIM



How to read this

- **x4plus INT8** differs from the shipped model by -0.00 dB (clean) and +0.05 dB (degraded) mean Y-PSNR. PSNR alone under-reports GAN-style texture differences; judge banding, smooth gradients and skin in the crops above.
- **general-x4v3** differs from the shipped model by +0.28 dB (clean) and -0.09 dB (degraded) mean Y-PSNR. PSNR alone under-reports GAN-style texture differences; judge banding, smooth gradients and skin in the crops above.
- Three photos and no perceptual metric (LPIPS) or blind A/B test: treat any quality verdict here as preliminary, especially for INT8 (GAN-trained texture is the usual failure point).
- general-x4v3 is trained for real-world degradation but has weaker denoise/deblur capacity than x4plus; adopting it would be a product decision (a “fast” mode) rather than a silent swap.

7 Upscaling quality: original vs upscaled (shipped models)

7.1 Production-style runs (an upload with no ground truth)

Production-style run: original upload → ×4 (no ground truth; red box = zoomed region)

Original — 512×512 px



Real-ESRGAN ×4 — 2048×2048 px (drawn at the same display size)



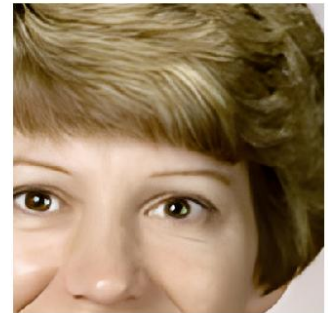
Original pixels, enlarged (nearest)



Bicubic ×4



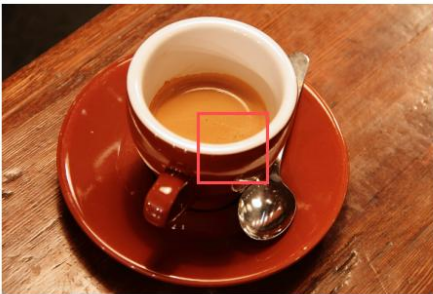
Real-ESRGAN ×4



Production-style run: original upload → ×2 (no ground truth; red box = zoomed region)

Real-ESRGAN ×2 — 1200×800 px (drawn at the same display size)

Original — 600×400 px



Original pixels, enlarged (nearest)



Bicubic ×2



Real-ESRGAN ×2



7.2 Against ground truth

×4, clean input

Image	LR → target	Bicubic	Lanczos	Real-ESRGAN	Sharpness ×GT	Time*
astronaut	128×128 → 512×512	25.71 dB / 0.835	25.82 dB / 0.835	24.04 dB / 0.801	0.03 → 0.89	2.9 s
coffee	150×100 → 600×400	26.06 dB / 0.750	26.11 dB / 0.751	26.38 dB / 0.750	0.01 → 0.08	2.4 s
chelsea	112×75 → 448×300	30.24 dB / 0.791	30.32 dB / 0.795	27.55 dB / 0.736	0.02 → 0.33	1.3 s
mean		27.34 dB / 0.792	27.42 dB / 0.793	25.99 dB / 0.762		

×2, clean input

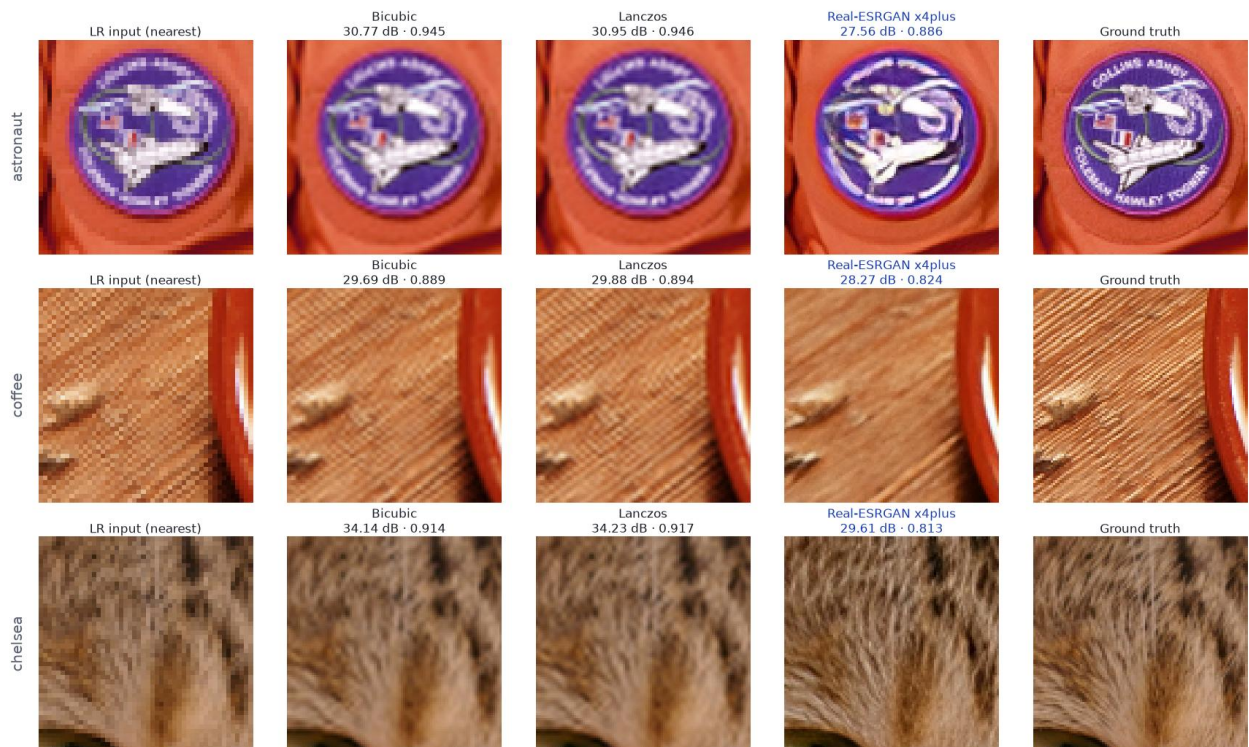
Image	LR → target	Bicubic	Lanczos	Real-ESRGAN	Sharpness ×GT	Time*
astronaut	256×256 → 512×512	30.77 dB / 0.945	30.95 dB / 0.946	27.56 dB / 0.886	0.18 → 1.02	2.7 s
coffee	300×200 → 600×400	29.69 dB / 0.889	29.88 dB / 0.894	28.27 dB / 0.824	0.10 → 0.19	2.7 s
chelsea	224×150 → 448×300	34.14 dB / 0.914	34.23 dB / 0.917	29.61 dB / 0.813	0.13 → 0.91	1.3 s
mean		31.53 dB / 0.916	31.69 dB / 0.919	28.48 dB / 0.841		

×4, degraded input (noise $\sigma=2$ + JPEG q50)

Image	LR → target	Bicubic	Lanczos	Real-ESRGAN	Sharpness ×GT	Time*
astronaut	128×128 → 512×512	24.27 dB / 0.719	24.24 dB / 0.713	22.72 dB / 0.740	0.02 → 0.85	2.7 s
coffee	150×100 → 600×400	24.89 dB / 0.659	24.87 dB / 0.656	24.89 dB / 0.677	0.01 → 0.07	2.5 s
chelsea	112×75 → 448×300	28.23 dB / 0.709	28.21 dB / 0.708	26.29 dB / 0.659	0.01 → 0.33	1.3 s
mean		25.79 dB / 0.695	25.78 dB / 0.692	24.63 dB / 0.692		

Cells: Y-PSNR / SSIM vs the original. Sharpness = variance of the Laplacian ÷ that of the ground truth (1.0 = as sharp as truth), shown bicubic → Real-ESRGAN. *Engine time at 2 vCPU.

×2, clean input — same crops · captions: full-image Y-PSNR / SSIM



Reading these numbers

- Clean inputs, ×4: Real-ESRGAN is -1.34 dB vs bicubic in mean PSNR. Bicubic wins on PSNR here, which is expected: PSNR rewards smooth averages, while Real-ESRGAN is trained (GAN + perceptual loss) to produce crisp, plausible detail. Sharpness vs truth: 0.02× (bicubic) vs 0.44× (Real-ESRGAN).
- Degraded inputs, ×4: -1.16 dB — bicubic magnifies noise and JPEG blocking, Real-ESRGAN was trained on exactly that damage.
- PSNR/SSIM measure pixel fidelity, not perceived quality; no LPIPS or human study was run. Three small, natural photos is a small sample.

8 Colourisation quality

Colour photographs → grayscale → DDColor-Tiny → compared with the original colours. The last row is a real black-and-white photograph (this project's own test image), no ground truth.

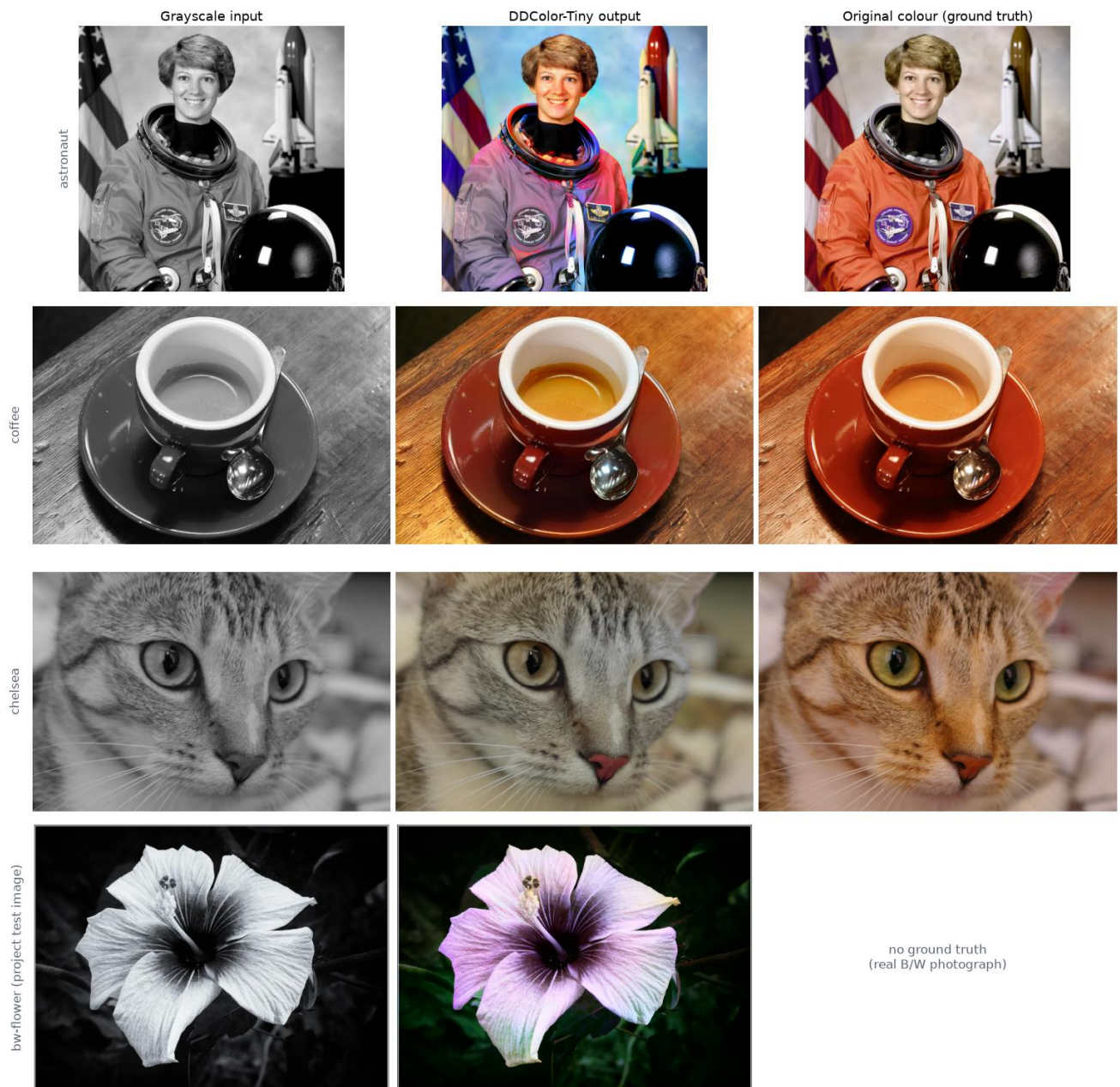


Image	Size	ΔE^*_{ab} (mean)	Baseline ΔE (no colour)	Colourfulness M (truth → output)	PSNR (RGB)	Time*
astronaut	512×512	27.8	20.6	72.6 → 69.9	16.3 dB	2.4 s
coffee	600×400	13.4	43.0	76.9 → 73.3	23.5 dB	2.3 s
chelsea	448×300	16.8	23.0	38.0 → 17.6	22.3 dB	2.3 s
mean		19.3	28.9			
bw-flower (real B/W)	1140×855	—	—	— → 26.8	—	2.2 s

*Engine time at 2 vCPU (warm). Lower ΔE is better. Colourisation is under-determined (a red and a blue car share a luminance), so a non-zero error with realistic, slightly desaturated colours is expected. Where the model is worse than “no colour” (astronaut), it chose plausible but wrong hues — for the astronaut it painted the orange suit blue/pink — so treat colourisation as a creative suggestion, not a restoration.

9 Recommendations for the Hostinger KVM 2 and limitations

- Keep **MAX_AI_WORKERS = 1** and 2-CPU container limits. Even the largest accepted job needs well under 8 GB, so on this plan CPU time, not memory, is the constraint.
- Size timeouts from the model: $t \approx \text{MAC} \div G$ with $G \approx 76 \text{ GMAC/s}$ measured here (§4.2); the largest accepted $\times 4$ job is $\approx 15 \text{ m } 04 \text{ s}$. **Re-measure G on the actual VPS** (one $\times 4 \text{ } 256^2$ job) and rescale: a shared vCPU is usually slower than a pinned core here.
- Use the memory model (§5.3) for admission control: estimate a job's peak before starting it and refuse what cannot fit.
- **Arena + memory-pattern ON** was -26 % time for +59 % RAM on the 600×400 job and -30 % time / 3262 MiB on $1024^2 \rightarrow 4096^2$. That fits an 8 GB plan with one worker, so it is the cheapest speed-up available, provided the memory model is re-fitted for it and the admission check stays on.
- **Constant-shape tiles + arena + shrinkage** on $1024^2 \rightarrow 4096^2$: -21 % time and +21 % RAM (1386 MiB) — the arena's memory stays bounded (arena alone: see the previous bullet). On the small 600×400 image the same option was +38 % because edge tiles are recomputed (§5.1b). So the best setting depends on image size: use it for large images and keep the shipped tiling for images that are only a few tiles wide.
- **Not worth changing:** constant-shape tiles (extra edge compute, §5.1b), thread-spinning, thread env variables, ORT 1.23.0, tile size 256 vs 384/512 — all within noise or worse. Stock ONNX Runtime defaults are clearly worse than the shipped settings.
- **general-x4v3** is $16\times$ faster and uses 332 MiB (§6). It is the only step-change available; it needs a blind A/B on your real uploads before becoming a default “fast” mode (keep x4plus as “best quality”).
- **INT8 x4plus** is $1.5\times$ faster with unchanged mean PSNR/SSIM on three photos and a $4\times$ smaller file (§6): low-risk to trial (same architecture), but calibration used 4 small images — recalibrate on representative uploads and check gradients by eye first.

Limitations

- Timings come from a Ryzen 9 8945HS (a fast laptop CPU) emulating 2 vCPUs with a quota and core pinning. A VPS vCPU may be an SMT thread on a slower shared core: the MAC counts, memory model and the *relative* effects are portable, absolute seconds are not.
- 1–3 runs per point (one for most ablations); differences under the measured noise are not significant. Other host activity was low but not zero.
- Quality uses three public-domain / CC0 photos ($\leq 600 \text{ px}$): NASA astronaut portrait, and coffee / chelsea from scikit-image; plus the project's own B/W flower photo. No LPIPS, no human study.
- ONNX exports of DDColor and Real-ESRGAN are third-party (DECISIONS.md); nothing here claims accuracy relative to the original PyTorch checkpoints.
- INT8 and general-x4v3 were built for this comparison only and are not part of the repository's shipped models.

Reproduce

```
# WORK/imgs/{astronaut,coffee,chelsea}.png = skimage.data samples; WORK/imgs/calib = held-out INT8 calibration images
python benchmarks/make_quality_inputs.py WORK
python benchmarks/run_benchmarks.py prep|baseline|ablation|ablation_big --work WORK
python benchmarks/build_candidates.py --work WORK
python benchmarks/run_benchmarks.py candidates|quality --work WORK
python benchmarks/make_report.py --work WORK --macs benchmarks/results/onnx_macs.json --out report.pdf
```